Topics, Partitions and Offsets

how a stream scales across machines

Phase 4 videos on YouTube

In one line: a topic is split into partitions, each an ordered append-only log stored on (and replicated across) brokers; records with the same key go to the same partition, offsets number records within a partition, and the partition count sets the maximum consumer parallelism.

1 One topic, many machines

A Kafka topic called `orders` receives 50,000 events per second. No single machine could store and serve that alone. So Kafka splits each **topic** into **partitions**, spread across **brokers**. Each partition is an independent, ordered, append-only log; together they hold the topic's data and its load.

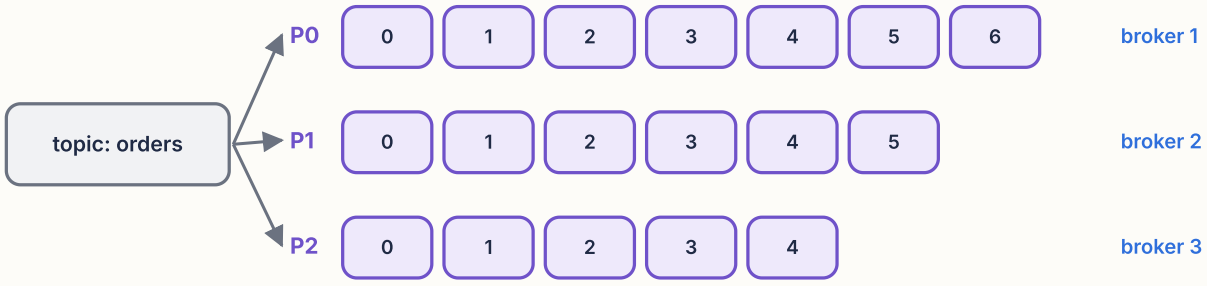


Figure 1. Three partitions of one topic on three brokers. Each grows independently.

2 Offsets

Within a partition, every record gets an **offset**: 0, 1, 2... Offsets are only meaningful inside their partition; there is no global order across partitions. A consumer's progress is a set of (partition, offset) pairs, and replay means moving those offsets back. Records stay until **retention** (time or size) removes old segments, or **compaction** keeps only the latest record per key.

3 Keys decide the partition

The producer picks a partition: with a key, it's `hash(key) % partitions`, so all records with the same key land in the same partition, in order. Without a key, records are spread (sticky batches in recent clients) for even load but with no ordering between them.

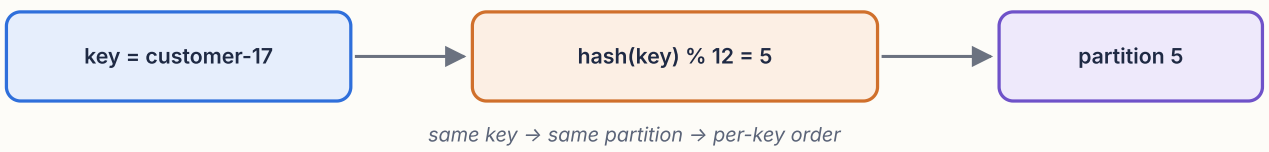


Figure 2. Keying by customer keeps each customer's events in order.

WORKED EXAMPLE: SIZING THE ORDERS TOPIC

The orders topic needs 50 MB/s at peak. Tests show one partition handles about 10 MB/s on the producer side, and one consumer about 5 MB/s of processing. Consumers are the bottleneck: 50 ÷ 5 = 10 partitions, rounded up to 12 for growth and an even spread over 3 brokers. Records are keyed by `customer_id`, so one customer's orders stay in order. A hot key (one huge customer) would still land in one partition, which is a limit to plan for.

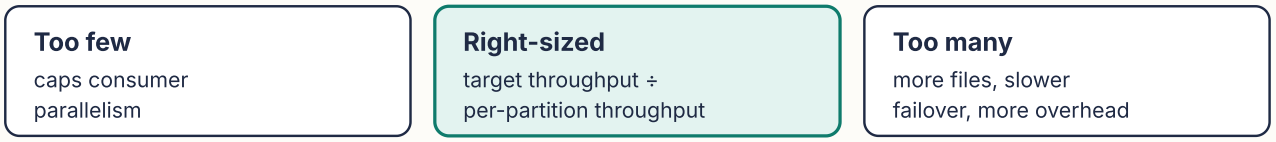


Figure 3. Choosing a partition count.

4 Changing partition counts

You can add partitions later, but the key-to-partition mapping changes for many keys, so per-key order is broken during the transition. Plan the count up front for a year or two of growth. You can't remove partitions without recreating the topic.

COMMON MISTAKES

- One partition 'to keep order', which caps throughput at one consumer.
- Thousands of partitions per topic 'just in case'.
- No key, then expecting per-entity ordering.
- Adding partitions to a keyed topic without considering ordering.

5 Key terms

Topic: named stream of records

Partition: ordered log, unit of parallelism and placement

Offset: record position within a partition

Broker: server storing partitions

Retention: how long or how much data is kept

Log compaction: keep only the latest record per key

6 Interview questions

Q1. Why are Kafka topics partitioned? BASIC

To spread storage and load across brokers and let several consumers read in parallel; each partition is an ordered log.

Q2. Is order guaranteed across a whole topic? BASIC

No, only within a partition. Use a key so related records share a partition.

Q3. How many partitions for a topic needing 100 MB/s, if one consumer handles 10 MB/s? SCENARIO

At least 10, plus headroom, say 12 to 16, so you can add consumers up to the partition count.

Q4. One partition has far more traffic than others. Why, and what do you do? SCENARIO

A hot key. Use a better key, add a suffix to spread a hot key (losing strict order for it), or handle that key separately.

Q5. What happens to ordering if you add partitions? TRICKY

The hash-to-partition mapping changes for many keys, so new records for a key may go to a different partition and be processed out of order with older ones.

Q6. What's log compaction good for? TRICKY

Topics that represent the latest state per key, like a changelog or configuration; consumers can rebuild current state without keeping every historical update.

Next: SD-I-005 · Consumer Groups and Rebalancing backendroadmap.com